

ATLAS Web Access Server^{0.0.1} OAS3

/openapi.json

Purpose

This document summarizes the features of ATLAS's "Web Access" server and provides examples to demonstrate how this server should be used. The ATLAS Web Access server was put in place to make it easy to access the ATLAS control system from the web while allowing the operators to monitor and control all access going through this server.

Errors

Several situations can occur that would cause the server to return an error. When one of these occurs, the server will return an error code. In all cases, the server will respond with a JSON file that includes an error message to indicate what went wrong. There error codes returned follow the [standard HTML responses](#).

- Code 200 Success
- Code 400 Bad Request. Server cannot understand request
- Code 403 Not authorized to access channel
- Code 404 Requested item was not found
- Code 500 Internal Server Error (Contact controls group)
- Code 503 Service unavailable. (Contact controls group)

Here is an example of how to read this error message:

```
BASE_URL = "http://146.139.66.20:8042/"
data = [{"device": "STP201", "chan": "CONTROL_X"},
        {"device": "STP201", "chan": "MONITOR_X"}]
reply = get(BASE_URL, json=data)
if(200 != reply.status_code):
    print("Code:", reply.status_code)
    if(hasattr(reply, 'json')):
        print(reply.json())
    raise RuntimeError("Failed to read from server")
```

Using the Server

Device Type	Accessible Channels
ATTENUATOR	IN,OUT,
FARADAYCUP	IN,OUT,CONTROL,SELECT_SIGNAL,DISPLAY_SIGNAL,
GAUGE	READOUT,
MAGNET	CONTROL,MONITOR,CONTROL_GAUSS,MONITOR_GAUSS,CONTROL_MODE,

MEASUREMENT	READ,
PROFILE_MONITOR	DISPLAY,ACTIVATE,
QUAD_DOUBLET	CONTROL_X,MONITOR_X,CONTROL_Y,MONITOR_Y,
QUAD_SINGLET	CONTROL_X,MONITOR_X,
QUAD_TRIPLET	CONTROL_X,MONITOR_X,CONTROL_Y,MONITOR_Y,CONTROL_2,MONITOR_2,
RESONATOR	
SLITS	TOP_CONTROL,TOP_MONITOR,BOTTOM_CONTROL,BOTTOM_MONITOR,LEFT_CONTROL,LEFT_MONITOR,RIGHT_CONTROL,RIGHT_MONITOR,
SOLENOID	
SOURCE	
STEERER	CONTROL_X,MONITOR_X,CONTROL_Y,MONITOR_Y,
STEPPER_ATTENUATOR	POSITION,
STEPPER_DIAGNOSTIC	
VALVE	OPEN,CLOSED,

Examples

Get Device Values

To get device values, send a JSON structure to the basic URL for the server (the URL that you used to view this documentation, but without the 'docs'). The JSON file schema should include all of the required information as shown below under "GET".

The 'chan' parameter is allowed to be any value in the table above. Note that this value changes depending on the type of device that is being accessed.

The JSON structure returned by the server will be the same as the one that was sent, but it will include the value ('val'), the timestamp ('ts') that the value was changed, and the name of the VSystem channel that it pulled the information from. The VSystem channel name can be used to validate the data is coming from the expected source as well as compare values from older servers. A timestamp will be zero if the timestamp feature is not enabled on the channel requested.

```
from requests import get
BASE_URL = "http://146.139.66.20:8042/"
data = [{"device": "STP201", "chan": "CONTROL_X"},
        {"device": "STP201", "chan": "MONITOR_X"}]
reply = get(BASE_URL, json=data)
print(reply.json())
```

Output

```
{
  'message': 'Items processed successfully',
  'data': [
    {'device': 'STP201', 'chan': 'CONTROL_X',
     'vsys': 'GUIDE_VDB::STP201:CONTROL_X',
     'val': -2.4100000858306885,
     'ts': 0},
    {'device': 'STP201', 'chan': 'MONITOR_X',
     'vsys': 'GUIDE_VDB::STP201:MONITOR_X',
     'val': 0.0,
     'ts': 0}
  ]
}
```

Put Device Values

To control a device through the server, send a JSON structure to the basic URL for the server (the URL that you used to view this documentation, but without the 'docs'). The JSON file schema should include all the required information as shown below under "PUT". Note that this is the same schema with the addition of a value ('val'). If the value requested is unfit for the channel (a string value is sent to an integer channel), then an error will occur.

The 'chan' parameter is allowed to be any value in the table above. Note that this value changes depending on the type of device that is being accessed.

The JSON structure returned by the server will be the same as the one that was sent, but it will include the value ('val'), the timestamp ('ts') that the value was changed, and the name of the VSystem channel that it pulled the information from. The VSystem channel name can be used to validate the data is coming from the expected source as well as compare values from older servers.

```
from requests import get
BASE_URL = "http://146.139.66.20:8042/"
data = [{"device":"STP201","chan":"CONTROL_X","val":1.5},
        {"device":"STP201","chan":"CONTROL_Y","val":1.6}]
reply = post(BASE_URL, json=data)
print(reply.json())
```

Output

```
{
  'message': 'Items processed successfully',
  'data': [
    {'device': 'STP201', 'chan': 'CONTROL_X',
     'vsys': 'GUIDE_VDB::STP201:CONTROL_X',
     'val': 1.5},
    {'device': 'STP201', 'chan': 'CONTROL_Y',
     'vsys': 'GUIDE_VDB::STP201:CONTROL_Y',
     'val': 1.6}
  ]
}
```

View Devices on Beam path

If all the devices on the current beam path for an experiment are desired, these can be viewed through this request. For the request, you must specify a device type that you want to view. Eligible device types are listed in the table above. This request does not require any JSON formatting for the request, but it includes a JSON file in the reply. For all devices in the beam path, use the wildcard "*". This method can also be accessed through a web browser directly.

Please refer to the ATLAS Automation Module Gitlab for more details on how to use Python to interface with this server.

Note on this Documentation

The documentation below is updated automatically when new features are added to the server.

[ATLAS Controls Group - Website](#)

[Apache 2.0](#)

default



GET / Read Atlas Device Value



Read the value of all requested channels from ATLAS Control System. JSON request must be formatted in a list of dictionaries with the schema below.

Parameters

Try it out

No parameters

Request body **required**

application/json

Example Value Schema

```
[
  {
    "device": "string",
    "chan": "string"
  }
]
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }]}</pre>	No links

POST / Write Atlas Device Value

Write the value of all requested channels into the ATLAS Control System. JSON request must include a list of dictionaries with the schema below.

Parameters[Try it out](#)

No parameters

Request body required

application/json

Example Value Schema

```
[
  {
    "device": "string",
    "chan": "string",
    "val": "string"
  }
]
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links
422	Validation Error Media type application/json Example Value Schema	No links

Code	Description	Links
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }]}</pre>	

GET

/info/{typeName} Get Device List

^

Get a list of all devices on the beamline. Acceptable device types are listed in the table above. Use "" for all devices.

Parameters

Try it out

Name	Description
typeName * required string (path)	typeName

Responses

Code	Description	Links
200	Successful Response	No links

Media type

Controls Accept header.

Code	Description	Links
	Example Value Schema	
	<code>"string"</code>	
422	Validation Error	No links
	Media type	
	application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>	

Schemas




```

HTTPValidationError {
  detail
    Detail [
      title: Detail
      ValidationError {
        loc*
          Location [
            title: Location
            {
              anyOf ->
                string
                integer
            }
          ]
        msg*
        type*
        string
        title: Message
        string
        title: Error Type
      }
    ]
  }
}

```

```

ReadValue {
  description: JSON schema that is required to read a value from the control system. For multiple values, seperate each dictionary by commas.
  device*
  chan*
  string
  title: Device
  string
  title: Chan
}

```

```

ValidationError {
  loc*
    Location [
      title: Location
      {
        anyOf ->
          string
          integer
        }
      ]
  msg*
  type*
  string
  title: Message
  string
  title: Error Type
}

```

```
WriteValue {  
  description: JSON schema that is required to write a value to the control system. For multiple values, seperate each dictionary by commas.  
  device* string  
             title: Device  
  chan* string  
          title: Chan  
  val* string  
        title: Val  
}
```